Proyecto Final · Data Scientist

Predicción de resultados ATP con Machine Learning

Modelado predictivo pre-partido sobre datos históricos del circuito ATP (1991–2024) mediante Random Forest, XGBoost y Regresión Logística. Visualización de datos con Python.

AUTOR
Andrés Pastor López

CURSO
Módulo de Big Data y Business Analysis

FECHA DE ENTREGA
25 de Abril de 2026

Dataset: `JeffSackmann/tennis_atp` · 1991–2024 · ~104k partidos Tipología: Data Scientist

Índice

1. Introducción 3
1.1 Contexto y motivación del proyecto 3
1.2 Objetivos 4
1.3 Alcance y limitaciones 4
1.4 Fuente de datos 5
1.5 Uso de herramientas de IA 5

PIPELINE ML

2. Análisis exploratorio de datos (EDA) 6
2.1 Descripción del dataset 6
2.2 Distribución de variables clave 7
2.3 Patrones y correlaciones 8
2.4 Valores nulos y calidad del dato 10

3. Preprocesamiento de datos 14
3.1 Carga y consolidación del dataset 14
3.2 Limpieza: nulos, duplicados y outliers 15
3.3 Eliminación de variables con data leakage 16
3.4 Winrate bayesiano con ventana deslizante 17
3.5 Simetrización del dataset 18
3.6 Feature engineering: variables de diferencia 19
3.7 Encoding y escalado 20

4. Selección y entrenamiento del modelo 21
4.1 Estrategia de split temporal 21
4.2 Regresión Logística (baseline) 22
4.3 Random Forest con Grid Search 22
4.4 XGBoost con Grid Search 23

5. Evaluación del modelo 24
5.1 Métricas de evaluación 24
5.2 Comparativa de modelos 25
5.3 Feature importance y análisis de coeficientes 26
5.4 Curvas ROC 27
5.5 Selección del modelo final 27

VISUALIZACIÓN PYTHON

6. Visualización de datos (Python) 28
6.1 Objetivos de la visualización 28
6.2 Arquitectura del código 28
6.3 Preprocesamiento para visualización 29
6.4 Estructura del dashboard 30

GESTIÓN Y CIERRE

7. Gestión del proyecto 31
7.1 Metodología aplicada 31
7.2 Alcance, tiempo y coste 32

8. Conclusiones y recomendaciones 33
8.1 Conclusiones sobre el modelo 33
8.2 Conclusiones sobre el pipeline 34
8.3 Recomendaciones para iteraciones futuras 35

9. Referencias 36

10. Anexos 37
10.1 Notebook completo (pipeline Python) 37
10.2 Módulo de visualización 37
10.3 Prompts e IA utilizados en el proyecto 37

1. Introducción

1.1 Contexto y motivación del proyecto

El deporte profesional genera cantidades masivas de datos estructurados: resultados, estadísticas de rendimiento, rankings, condiciones de juego. Durante décadas esos datos se usaron principalmente para crónicas y clasificaciones. Hoy son la materia prima de modelos que intentan anticipar lo que va a pasar antes de que ocurra.

Este proyecto nace del interés por una pregunta concreta: ¿hasta qué punto puede un modelo de machine learning predecir el resultado de un partido de tenis profesional usando solo información disponible antes de que empiece? No estadísticas del propio partido, no datos recopilados durante el juego, solo lo que se sabe de antemano: el ranking de los jugadores, la superficie, el torneo, el historial reciente.

La elección del tenis ATP como dominio de aplicación responde a varias razones prácticas. El circuito masculino dispone de registros históricos completos, fiables y públicos que se remontan décadas atrás. Es un deporte individual, lo que elimina la complejidad de modelar dinámicas de equipo. Y existe un referente claro contra el que medir el rendimiento: el propio sistema de predicción de la ATP, que según datos públicos alcanza una accuracy aproximada del 76%.

Antes de decantarse por este proyecto se valoró otra línea relacionada con el deporte: un modelo de predicción de lesiones en jugadores de fútbol. Se descartó por falta de datos públicos con la granularidad necesaria. El tenis, en cambio, cuenta con el repositorio de Jeff Sackmann, que recoge cada partido del circuito ATP principal desde 1968 en formato estructurado y de acceso libre.

El resultado de este trabajo no pretende competir con los sistemas propietarios de las organizaciones deportivas, que tienen acceso a variables que no están en ningún dataset público. El objetivo es técnico y formativo: construir un pipeline completo de ciencia de datos, desde la carga y limpieza del dato crudo hasta el entrenamiento y evaluación de varios modelos, y documentar con rigor cada decisión tomada en el proceso.

1.2 Objetivos

El objetivo principal del proyecto es desarrollar un modelo de clasificación binaria capaz de predecir el resultado de un partido ATP utilizando exclusivamente información pre-partido.

Los objetivos específicos son los siguientes:

- Construir un pipeline reproducible de carga, limpieza y preprocesamiento sobre el dataset histórico ATP (1991–2024).
- Diseñar features predictoras que capturen el estado de forma reciente de cada jugador sin introducir data leakage.
- Entrenar y comparar tres modelos de clasificación: Regresión Logística como baseline, Random Forest y XGBoost.
- Evaluar el rendimiento de cada modelo mediante métricas adecuadas para clasificación binaria: accuracy, AUC-ROC y F1-score.

— Aproximarse al umbral de rendimiento del sistema de predicción oficial de la ATP (~76% de accuracy) como referencia externa de calidad.
— Visualizar los resultados del análisis y del modelo en un dashboard de Power BI orientado a la interpretación de los datos.

1.3 Alcance y limitaciones

El proyecto cubre el circuito ATP Tour principal masculino entre 1991 y 2024. Quedan fuera del alcance los torneos de cualificación, los challengers, los futures y el circuito femenino WTA.

El modelo trabaja exclusivamente con información disponible antes del inicio del partido. Esta decisión es deliberada: incluir estadísticas generadas durante el juego —aces, dobles faltas, puntos de saque— introduciría data leakage y produciría un modelo sin aplicación real. El tratamiento detallado de esta decisión se documenta en la sección de preprocesamiento.

Existen limitaciones inherentes al dataset y al enfoque adoptado:

— El modelo no tiene acceso a información sobre el estado físico de los jugadores, lesiones recientes, fatiga acumulada ni condiciones climatológicas del día del partido.
— Las cuotas de apuestas y los mercados de predicción, que a menudo incorporan información cualitativa difícil de cuantificar, no forman parte del modelo.
— Los partidos con datos de ranking faltantes se han eliminado del dataset, lo que puede sesgar ligeramente la muestra hacia torneos y épocas con mejor cobertura de datos.
— El rendimiento del modelo puede degradarse en partidos con jugadores muy jóvenes o recién incorporados al circuito, dado que su historial es limitado.

1.4 Fuente de datos

Los datos provienen del repositorio público de Jeff Sackmann, disponible en GitHub:

`https://github.com/JeffSackmann/tennis_atp`

El repositorio recoge cada partido del circuito ATP en archivos CSV anuales. Cada fila representa un partido e incluye información sobre el torneo, la superficie, la ronda, los dos jugadores, sus rankings y puntos de ranking en el momento del partido, sus estadísticas físicas y las estadísticas técnicas del propio encuentro.

Para este proyecto se han cargado los archivos correspondientes al período 1991–2024, excluyendo los ficheros de categorías menores. El dataset consolidado antes de limpieza contiene 108.498 partidos y 49 columnas. Tras el preprocesamiento completo, el dataset de trabajo queda en 104.176 partidos y 30 columnas.

```
Dataset:    JeffSackmann/tennis_atp  (GitHub, licencia MIT)
Período:   1991 – 2024
Partidos:  108.498 brutos  →  104.176 tras limpieza
Columnas:  49 originales  →  30 tras preprocesamiento
```

1.5 Uso de herramientas de IA

De acuerdo con los criterios de evaluación del CEI, se documenta a continuación el uso de herramientas de inteligencia artificial durante el desarrollo de este proyecto.

Se emplearon los modelos Claude Sonnet 4.6 y Claude Opus 4.6 de Anthropic, accedidos a través de las interfaces Claude.ai y Claude Code. Su uso se limitó a las siguientes tareas:

- **Corrección de errores en el código:** detección y resolución de bugs en el pipeline de Python, especialmente en las fases de imputación, simetrización y escalado.
- **Comentado y documentación del código:** revisión y mejora de los comentarios en línea y docstrings de las funciones desarrolladas, siguiendo las convenciones PEP8.
- **Apoyo en la redacción de la memoria:** soporte para estructurar y revisar el texto de este documento.

En ningún caso se utilizó la IA para tomar decisiones de diseño del modelo, interpretar resultados, seleccionar features ni evaluar el rendimiento de los algoritmos. Todas las decisiones técnicas y analíticas fueron tomadas por el autor.

A continuación se recogen ejemplos representativos de los prompts utilizados:

Herramienta: Claude Sonnet 4.6 / Opus 4.6 (Anthropic)

Corrección de código:
"El merge del winrate bayesiano produce NaN en algunas filas tras la simetrización.
Revisa el código e identifica dónde se pierde la asignación."

Comentado de código:
"Añade comentarios PEP8 a esta función de simetrización explicando cada paso."

Redacción de memoria:
"Redacta la sección de preprocesamiento de datos siguiendo este esquema [...]."

2. Análisis exploratorio de datos (EDA)

Antes de construir ningún modelo, es necesario entender qué hay en el dataset: su estructura, sus distribuciones, sus valores faltantes y las relaciones entre variables. Este capítulo recorre ese proceso de forma sistemática, desde una primera inspección del dato crudo hasta el análisis de los patrones más relevantes para la predicción.

2.1 Descripción del dataset

El dataset consolidado a partir del repositorio de Jeff Sackmann cubre 34 temporadas del circuito ATP principal, desde 1991 hasta 2024. Cada fila representa un partido individual con información del torneo, los jugadores y el resultado.

Dimensión	Valor
Partidos (filas)	108.375
Variables (columnas)	49
Período cubierto	1991 – 2024
Temporadas	34
Tipos de columna	object (str): 11 · int64: 5 · float64: 33

Tabla 2.1 — Dimensiones del dataset original antes de cualquier transformación.

Las 49 columnas originales se pueden agrupar en cuatro bloques funcionales:

- **Identificación del torneo:** tourney_id, tourney_name, surface, draw_size, tourney_level, tourney_date, match_num.
- **Información del ganador:** winner_id, winner_seed, winner_entry, winner_name, winner_hand, winner_ht, winner_ioc, winner_age, winner_rank, winner_rank_points.
- **Información del perdedor:** columnas equivalentes prefijadas con loser_.
- **Estadísticas del partido:** score, best_of, round, minutes y 19 columnas de estadísticas técnicas in-match (aces, dobles faltas, puntos de saque, break points).

Una observación importante sobre la estructura: el dataset tiene un sesgo de diseño que hay que corregir antes de modelar. Todas las filas registran el resultado desde la perspectiva del ganador, lo que significa que la variable objetivo está implícita en la posición de las columnas, no en un campo explícito. Esta asimetría se resuelve en la fase de preprocesamiento mediante la simetrización del dataset.

2.2 Distribución de variables clave

A continuación se analiza la distribución de las variables más relevantes para el modelo.

Superficie de juego

Superficie	Partidos · % del total
Hard (pista dura)	54.581 · 50,4%
Clay (tierra batida)	35.972 · 33,2%
Grass (hierba)	10.539 · 9,7%
Carpet (moqueta)	7.230 · 6,7%

Tabla 2.2 — Distribución de partidos por superficie.

La pista dura concentra la mitad del volumen total. La moqueta, prácticamente desaparecida del circuito moderno desde principios de los 2000, representa menos del 7% y corresponde casi en su totalidad a partidos anteriores a 2005.

Nivel de torneo

Código · Categoría	Partidos · % del total
A · ATP 250 / ATP 500	62.000 · 57,2%
M · Masters 1000	18.749 · 17,3%
G · Grand Slam	17.145 · 15,8%
D · Copa Davis	9.845 · 9,1%
F · ATP Finals	572 · 0,5%
O · Otros	64 · 0,1%

Tabla 2.3 — Distribución de partidos por nivel de torneo.

Los torneos ATP 250/500 son la categoría más numerosa por volumen. Los Grand Slams representan el 15,8% de los partidos y tienen una característica que los diferencia del resto: se juegan al mejor de cinco sets en lugar de tres, lo que puede influir en la capacidad predictiva del modelo en esa categoría.

Mano dominante

Mano	Jugadores ganadores
Diestro (R)	92.928 · 85,7%
Zurdo (L)	14.973 · 13,8%
Desconocida (U)	459 · 0,4%
Ambidiestro (A)	15 · 0,0%

Tabla 2.4 — Distribución por mano dominante del ganador.

Los valores U (unknown) y A (ambidextrous) son residuales y se eliminan durante el preprocesamiento por no aportar información fiable.

Estadísticas descriptivas de variables numéricas clave

Variable	Media	Mediana	Rango
winner_rank	80,65	46	1 – 2.101
loser_rank	118,85	70	1 – 2.159
winner_rank_points	1.445	885	1 – 16.950
loser_rank_points	907	665	1 – 16.950
winner_age	25,79 años	25,5 años	14,3 – 44,6
loser_age	25,91 años	25,7 años	14,5 – 44,0
winner_ht	185,83 cm	185 cm	3 – 211 cm*
loser_ht	185,28 cm	185 cm	3 – 211 cm*

Tabla 2.5 — Estadísticas descriptivas. (*) Los valores mínimos de altura son errores de registro corregidos en preprocesamiento.

Dos observaciones relevantes. Primero, la diferencia entre el ranking medio del ganador (80,65) y el del perdedor (118,85) confirma que el ranking es un predictor potente: el jugador mejor clasificado gana con más frecuencia. Segundo, los mínimos de altura de 3 cm son errores claros de registro que se filtran en la limpieza.

2.3 Patrones y correlaciones

El hallazgo más importante del EDA es la relación entre la diferencia de ranking y el resultado del partido. Sobre los 105.158 partidos con ranking disponible para ambos jugadores, el favorito —definido como el jugador con menor número de ranking— gana el 65,2% de los encuentros.

Tasa de victorias del favorito (menor ranking): 65,2%
Upsets (gana el jugador peor clasificado): 34,8%
Partidos con ranking disponible: 105.158

Este 65,2% es el punto de partida del modelo. Cualquier clasificador que no supere esa cifra es equivalente a predecir siempre al favorito, lo que no requiere ningún modelo. El objetivo es superar ese umbral de forma consistente.

Tasa de upsets por superficie

Superficie	Tasa de upsets · Partidos
Hard (pista dura)	34,2% · 52.916
Clay (tierra batida)	35,6% · 34.937
Grass (hierba)	35,1% · 10.383
Carpet (moqueta)	36,0% · 6.886

Tabla 2.6 — Tasa de upsets por superficie. La diferencia entre superficies es reducida (~2 puntos).

Las diferencias entre superficies son pequeñas, con un rango de apenas dos puntos porcentuales. La moqueta y la tierra batida muestran una tasa de upsets ligeramente superior, lo que sugiere que en esas superficies el ranking explica algo menos el resultado. La hierba, asociada popularmente con sorpresas, tiene una tasa similar a la pista dura.

Tasa de upsets por ronda

Ronda	Tasa de upsets · Partidos
R128 (primera ronda)	34,2% · 10.452
R64	35,7% · 16.253
R32	35,8% · 34.711
R16 (octavos)	34,0% · 18.489
QF (cuartos)	35,1% · 9.293
SF (semifinal)	37,7% · 4.747
F (final)	37,0% · 2.418

Tabla 2.7 — Tasa de upsets por ronda. Las eliminatorias avanzadas muestran más imprevisibilidad.

Las semifinales y finales registran las tasas de upset más altas (37–38%). Esto es coherente con el diseño del torneo: en las rondas finales los jugadores tienen rankings más cercanos entre sí, lo que reduce la capacidad predictiva del ranking como variable aislada.

Evolución temporal del volumen de partidos

El dataset presenta una anomalía clara en 2020: solo 1.462 partidos registrados frente a los 2.700–3.900 habituales de una temporada normal. Este descenso se explica por la suspensión del circuito ATP entre marzo y agosto de 2020 a causa de la pandemia de COVID-19. Este dato no afecta a la calidad del modelo, pero sí tiene implicaciones en el cálculo del winrate bayesiano con ventana deslizante, ya que la ventana de 30 partidos puede cruzar el período de pausa.

Jugadores con mayor número de victorias en el período

Jugador	Victorias (1991–2024)
Roger Federer	1.265
Novak Djokovic	1.139
Rafael Nadal	1.091
Andy Murray	748
David Ferrer	740
Andre Agassi	692
Pete Sampras	687
Tomas Berdych	643
Lleyton Hewitt	619

Yevgeny Kafelnikov	613

Tabla 2.8 — Top 10 jugadores por victorias en el circuito ATP principal, 1991–2024.

2.4 Valores nulos y calidad del dato

La auditoría de valores nulos revela tres patrones distintos que requieren tratamientos diferentes.

Primer patrón — nulos estructurales en variables de entrada al torneo

Variable	Nulos	% del total
winner_entry	95.004	87,7%
loser_entry	86.454	79,8%
loser_seed	84.167	77,7%
winner_seed	64.441	59,5%

Tabla 2.9 — Variables de entrada y cabeza de serie con alta proporción de nulos estructurales.

Estos nulos no son datos faltantes en sentido estricto: significan que el jugador no tenía cabeza de serie (seed) o que entró al torneo de forma directa sin vía especial (entry). Se tratan imputando 0 en las columnas de seed y "Direct" en las de entry.

Segundo patrón — nulos en bloque en estadísticas in-match

Bloque de columnas	Nulos · % del total
19 columnas de estadísticas in-match (aces, df, svpt...)	10.325 · 9,53% cada una
minutes	13.274 · 12,25%

Tabla 2.10 — Las 19 columnas de estadísticas in-match comparten exactamente el mismo número de nulos, lo que indica partidos sin datos técnicos registrados.

Las 19 columnas de estadísticas técnicas del partido tienen exactamente el mismo número de nulos: 10.325. Esto indica que son partidos completos sin datos registrados, no casos aislados. En cualquier caso, estas columnas se eliminan íntegramente del dataset por generar data leakage: son estadísticas producidas durante el partido, no disponibles antes de que empiece.

Tercer patrón — nulos en variables de ranking y físicas

Variable	Nulos	% del total
loser_ht	4.490	4,14%
loser_rank_points	3.567	3,29%
loser_rank	2.584	2,38%
winner_ht	2.248	2,07%

winner_rank_points	2.203	2,03%
winner_rank	1.215	1,12%
loser_age / winner_age	15 / 5	~0,01%

Tabla 2.11 — Variables de ranking, altura y edad con nulos en proporción reducida.

Las alturas se imputan con la mediana, resistente a outliers. Las edades, con muy pocos nulos, también se imputan con la mediana. Las filas con ranking nulo se eliminan directamente: el ranking es la feature predictora más importante del modelo y cualquier imputación introduciría ruido relevante en una variable central.

En resumen, el dataset original tiene buena cobertura en las variables clave. Los nulos más numerosos (seed y entry) son estructurales y tienen una interpretación directa. Los nulos en estadísticas in-match son irrelevantes porque esas columnas se eliminan por diseño. Y los nulos en ranking afectan a menos del 2,5% de los partidos, por lo que su eliminación no compromete la representatividad del dataset.

3. Preprocesamiento de datos

El preprocesamiento es la fase más extensa del proyecto. Partiendo de un dataset crudo con 108.375 partidos y 49 columnas, se aplica una secuencia de siete pasos que transforma el dato en un dataset listo para entrenar modelos: limpio, simétrico, sin data leakage y con las features adecuadas. Cada decisión se documenta con la justificación técnica correspondiente.

Pipeline de preprocesamiento — resumen de pasos

1. Carga y consolidación	108.375 partidos × 49 columnas
2. Limpieza (nulos, outliers)	104.163 partidos × 30 columnas
3. Eliminación in-match	–19 columnas (data leakage)
4. Winrate bayesiano	+2 columnas (feature engineering)
5. Simetrización	208.326 partidos × ~32 columnas
6. Variables de diferencia	+6 columnas
7. Encoding y escalado	dataset final listo para modelar

3.1 Carga y consolidación del dataset

El repositorio de Jeff Sackmann organiza los datos en archivos CSV anuales. La carga consiste en localizar los archivos relevantes dentro del repositorio clonado y concatenarlos en un único DataFrame.

Los criterios de filtrado son dos: año dentro del rango 1991–2024, y nombre de archivo que no contenga las palabras clave de categorías menores. Se excluyen explícitamente futures, qualifiers, challengers y dobles, ya que corresponden a circuitos distintos al ATP Tour principal. Los 34 archivos que superan ese filtro se leen individualmente y se concatenan en un único DataFrame con índice continuo, resultando en 108.375 filas y 49 columnas.

La columna tourney_date viene codificada como entero en formato YYYYMMDD. Se convierte inmediatamente a tipo fecha para poder operar con ella en el split temporal posterior y en el cálculo del winrate con ventana deslizante.

3.2 Limpieza: nulos, duplicados y outliers

La limpieza se aplica en tres fases. El criterio general es intervenir lo mínimo necesario: imputar solo cuando la ausencia de dato tiene una interpretación clara, y eliminar filas únicamente cuando la imputación introduciría ruido relevante en variables críticas.

Fase 1 — Imputación de variables de entrada al torneo

Los nulos en winner_seed y loser_seed no indican datos faltantes: significan que el jugador no era cabeza de serie. Se imputan con 0. Los nulos en winner_entry y loser_entry indican entrada directa al cuadro principal sin vía especial, por lo que se imputan con el valor Direct.

Variable	Nulos · Tratamiento aplicado
winner_seed / loser_seed	64.441 / 84.167 → Imputados con 0 (no seeded)
winner_entry / loser_entry	95.004 / 86.454 → Imputados con Direct

Tabla 3.1 — Imputación de variables de entrada. Los nulos son estructurales, no datos faltantes.

Fase 2 — Imputación de variables numéricas y categóricas

Las alturas y edades con nulos se imputan con la mediana de cada columna, por su resistencia a valores extremos. La columna surface, con solo 53 nulos sobre 108.375 filas, se imputa con la moda (Hard). La columna loser_hand, con 4 nulos, también se imputa con la moda (R, diestro).

Variable	Mediana / Moda aplicada
winner_ht / loser_ht	185 cm (mediana)
winner_age / loser_age	25,6 / 25,8 años (mediana)
surface	Hard (moda — 50,4% del dataset)
loser_hand	R — diestro (moda — 85,7%)

Tabla 3.2 — Valores de imputación para variables numéricas y categóricas.

Fase 3 — Eliminación de filas con ranking nulo y outliers de altura

Las filas con ranking nulo se eliminan directamente. Afectan a 4.199 partidos (3,9% del dataset). El ranking es la feature más importante del modelo: imputar valores de ranking que no existieron en el momento del partido introduciría ruido en la variable de mayor peso predictivo.

Los outliers de altura se detectan inspeccionando los mínimos de la columna: valores de 3 cm y 15 cm son imposibles en cualquier contexto y corresponden a errores de registro. Se aplica un filtro que retiene solo las filas dentro del rango fisiológicamente plausible para un tenista profesional, entre 165 cm y 215 cm. Esto elimina 13 filas, menos del 0,01% del dataset.

Operación	Filas eliminadas	Shape resultante
Dataset original	—	108.375 × 49
Eliminar columnas in-match	—	108.375 × 30
Dropna en columnas de ranking	4.199	104.176 × 30
Filtro outliers altura	13	104.163 × 30

Tabla 3.3 — Evolución del dataset durante la limpieza.

Tras la limpieza completa el dataset no contiene ningún valor nulo ni filas duplicadas. El resultado es un DataFrame de 104.163 partidos y 30 columnas.

3.3 Eliminación de variables con data leakage

Esta es la decisión de diseño más importante del preprocesamiento. El objetivo del modelo es predecir el resultado de un partido antes de que empiece, usando solo información disponible en ese momento. Cualquier variable que se genere durante el partido no puede ser una feature: conocerla en el momento de la predicción sería imposible en un entorno real.

El dataset original incluye 19 columnas de estadísticas técnicas del partido más la duración en minutos. Todas se generan durante el juego: aces, dobles faltas, puntos de saque, puntos ganados con el primer y segundo servicio, break points salvados y afrontados, y el número de games de servicio. Incluirlas produciría métricas de evaluación infladas que no se replicarían fuera del entorno de pruebas.

Las 20 columnas afectadas se eliminan en bloque al inicio del pipeline, antes de cualquier otra transformación. La columna score se elimina igualmente en fases posteriores por el mismo motivo: contiene el resultado detallado del partido, que por definición no está disponible antes de jugarlo. Tras esta eliminación el dataset pasa de 49 a 30 columnas.

3.4 Winrate bayesiano con ventana deslizante

La feature más elaborada del pipeline es el winrate bayesiano de cada jugador. El objetivo es capturar el estado de forma reciente sin que el cálculo filtre información futura hacia el pasado, lo que constituiría otro tipo de data leakage.

El winrate simple —victorias divididas por partidos totales— tiene dos problemas para este proyecto. Primero, usa toda la historia del jugador, incluyendo partidos jugados después del que se está prediciendo. Segundo, es inestable para jugadores con pocos partidos: un jugador con 2 victorias en 2 partidos tendría un winrate del 100%, idéntico al de Djokovic en su mejor temporada.

La solución combina dos mecanismos:

- **Ventana deslizante de 30 partidos:** para cada partido, solo se usan los 30 partidos más recientes del jugador anteriores a esa fecha. Esto garantiza que el cálculo refleja el estado de forma reciente y no contamina con información futura.
- **Bayesian average con prior = 5:** se añade un prior equivalente a 5 partidos con una tasa de victoria del 50%, lo que estabiliza el estimador cuando el historial disponible es corto. La fórmula aplicada es: winrate = (victorias_en_ventana + 5) / (partidos_en_ventana + 10), lo que equivale a asumir un prior de 5 victorias y 5 derrotas antes de observar cualquier dato real.

El cálculo se realiza sobre el dataset antes de la simetrización, para evitar que las filas duplicadas en el paso siguiente contaminen la ventana deslizante. La implementación usa un merge vectorizado sobre el dataset ordenado cronológicamente, evitando el uso de apply() para mantener tiempos de ejecución razonables sobre las 104.163 filas. El resultado son dos columnas nuevas, winner_winrate y loser_winrate, que recogen el estado de forma de cada jugador en el momento de cada partido. El dataset intermedio se guarda en Google Drive como punto de control antes de continuar con el pipeline.

3.5 Simetrización del dataset

El dataset original tiene un sesgo estructural: todas las filas representan el partido desde la perspectiva del ganador. La columna winner siempre corresponde al jugador que ganó. Si se entrenara un modelo directamente sobre este formato, aprendería a predecir siempre al jugador en la posición winner, lo que no tiene aplicación real.

La simetrización resuelve este problema duplicando cada partido en dos filas con perspectivas invertidas:

- **Fila A (result = 1):** player1 es el ganador original, player2 es el perdedor original.
- **Fila B (result = 0):** player1 es el perdedor original, player2 es el ganador original.

Todas las columnas se renombran de winner_ y loser_ a player1_ y player2_ de forma consistente en ambas copias. Las columnas de winrate se propagan correctamente: winner_winrate pasa a player1_winrate en la fila A y a player2_winrate en la fila B, y viceversa para loser_winrate.

Dataset antes de simetrizar: 104.163 partidos · result implícito en posición
Dataset después de simetrizar: 208.326 filas · result = 0 o 1 explícito
Balance de clases: 50% clase 1 · 50% clase 0 (por construcción)

El balance de clases perfecto al 50/50 es una consecuencia directa del diseño: cada partido genera exactamente una fila con result=1 y una con result=0. Esto elimina la necesidad de técnicas de rebalanceo como SMOTE o ajuste de pesos de clase, y simplifica la interpretación de las métricas de evaluación.

3.6 Feature engineering: variables de diferencia

Una vez simetrizado el dataset, se construyen seis variables de diferencia entre player1 y player2. La intuición es que la diferencia relativa entre los jugadores tiene más poder predictivo que los valores absolutos de cada uno: no importa tanto que player1 tenga ranking 50, sino que player1 tenga ranking 50 y player2 tenga ranking 120.

Variable creada	Fórmula
rank_diff	player1_rank − player2_rank
rank_pts_diff	player1_rank_points − player2_rank_points
age_diff	player1_age − player2_age
ht_diff	player1_ht − player2_ht
seed_diff	player1_seed − player2_seed
winrate_diff	player1_winrate − player2_winrate

Tabla 3.4 — Variables de diferencia. Un valor negativo en rank_diff indica que player1 está mejor clasificado.

La convención es consistente en todo el dataset: un valor negativo en rank_diff significa que player1 tiene menor número de ranking, es decir, está mejor clasificado. En las filas donde

result=1, el valor esperado de rank_diff es negativo, ya que el ganador tiende a tener mejor ranking. Esta es la relación principal que el modelo aprende a explotar.

Tras construir las variables de diferencia se eliminan las columnas absolutas originales que quedan redundantes, dejando el dataset con las features de torneo, superficie, variables de diferencia y la variable objetivo result.

3.7 Encoding y escalado

Los algoritmos de machine learning requieren que todas las variables sean numéricas. Las variables categóricas del dataset se tratan con dos estrategias según su naturaleza.

Label Encoding para variables ordinales

Las variables round y tourney_level tienen un orden natural que conviene preservar. Se aplica Label Encoding asignando un entero a cada categoría en orden ascendente de importancia competitiva: para las rondas, de R128 hasta la final; para el nivel de torneo, de los ATP 250 hasta los Grand Slams.

Variable	Mapping aplicado
round	R128=1 R64=2 R32=3 R16=4 QF=5 SF=6 F=7 (RR=0)
tourney_level	A=1 D=2 O=3 F=4 M=5 G=6

Tabla 3.5 — Label Encoding para variables ordinales. El orden refleja la progresión competitiva.

One-Hot Encoding para variables nominales

Las variables sin orden natural —surface, player1_hand, player2_hand, player1_entry, player2_entry— se transforman con One-Hot Encoding, creando una columna binaria por cada categoría. Se elimina una categoría de referencia en cada variable para evitar multicolinealidad perfecta. Las categorías minoritarias o anómalas en hand (valores ambidiestro y desconocido) y en entry (categorías con menos de 100 apariciones en el dataset) se eliminan antes del encoding por no aportar información estadísticamente relevante.

Escalado con StandardScaler

Los modelos de esta familia —en especial la Regresión Logística— son sensibles a la escala de las variables. Una diferencia de ranking de 500 posiciones y una diferencia de edad de 3 años no son comparables en magnitud sin normalización previa.

Se aplica StandardScaler exclusivamente sobre las columnas no binarias. El aspecto crítico es el orden de operaciones: el split temporal se realiza antes del escalado, y el scaler se ajusta únicamente sobre los datos de entrenamiento. Aplicarlo sobre el conjunto completo antes de dividir trasladaría información estadística de los datos de test al proceso de entrenamiento, introduciendo data leakage en la fase de normalización.

El corte temporal se establece en el 1 de enero de 2021: todo lo anterior forma el conjunto de entrenamiento, todo lo posterior es el conjunto de test. Esta estrategia replica la condición real de uso del modelo, que se entrena con el pasado y predice sobre partidos que aún no ha visto.

Conjunto	Partidos (sin simetrizar)	Filas (simetrizadas)
Train (< 2021)	92.601 · 88,9%	185.202
Test (≥ 2021)	11.562 · 11,1%	23.124
Total	104.163	208.326

Tabla 3.6 — Distribución train/test. El test cubre las temporadas 2021–2024.

El conjunto de test abarca cuatro temporadas completas: 2021, 2022, 2023 y 2024. Este período es suficientemente largo para evaluar la estabilidad del modelo ante cambios en el circuito, como la irrupción de nuevos jugadores o la evolución del ranking de los jugadores establecidos.

Dataset final tras preprocesamiento completo

Filas totales (simetrizadas): 208.326
Train: 185.202 (partidos < 2021)
Test: 23.124 (partidos 2021-2024)

Balance de clases: 50% / 50% (por construcción)
Valores nulos: 0
Duplicados: 0

4. Selección y entrenamiento del modelo

Con el dataset preprocesado, simetrizado y escalado, el pipeline entra en la fase de modelado. Se entrenan tres clasificadores con distintos niveles de complejidad, siguiendo una estrategia de baseline progresivo: partir de un modelo simple e interpretable, y añadir complejidad solo si produce mejoras reales en las métricas de evaluación.

Los tres modelos comparten el mismo split temporal, el mismo conjunto de features y la misma métrica principal de optimización: el AUC-ROC. Esta consistencia permite comparar sus resultados de forma directa y atribuir las diferencias de rendimiento a los algoritmos, no a variaciones en los datos de entrada.

4.1 Estrategia de split temporal

El split entre entrenamiento y test se realiza por fecha, no de forma aleatoria. Todo partido anterior al 1 de enero de 2021 forma el conjunto de entrenamiento; todo partido posterior o igual a esa fecha forma el conjunto de test. Esta decisión es fundamental para que la evaluación sea realista.

Un split aleatorio mezclaría partidos de distintas épocas en train y test, permitiendo que el modelo vea datos del futuro durante el entrenamiento. En un problema de predicción deportiva esto es especialmente grave: los jugadores, sus rankings y sus estados de forma evolucionan con el tiempo. Un modelo que se entrena con datos de 2023 y se evalúa con datos de 2019 no está siendo evaluado en ninguna condición real.

Conjunto	Partidos (sin simetrizar)	Filas (simetrizadas)
Train (< 2021)	92.601 · 88,9%	185.202
Test (≥ 2021)	11.562 · 11,1%	23.124
Total	104.163	208.326

Tabla 4.1 — Distribución train/test con corte temporal en enero de 2021.

La validación cruzada interna durante el Grid Search usa StratifiedKFold con 5 folds y shuffle=False. Mantener shuffle=False preserva el orden cronológico entre folds, de forma que el modelo nunca entrena con datos más recientes que los que usa para validar dentro de cada iteración.

4.2 Regresión Logística (baseline)

El primer modelo es una Regresión Logística, que actúa como baseline de referencia. Su función no es alcanzar el máximo rendimiento posible, sino establecer un suelo: si los modelos más complejos no superan este resultado, existe un problema en el diseño del pipeline o en las features, no en la elección del algoritmo.

Se configura con solver lbfgs, un máximo de 1.000 iteraciones y paralelización sobre todos los núcleos disponibles. No se aplica Grid Search: la Regresión Logística tiene pocos hiperparámetros relevantes y su valor en este contexto es precisamente la simplicidad, que permite aislar el efecto de las features del efecto de la optimización de hiperparámetros.

Una ventaja adicional de este modelo es la interpretabilidad directa de sus coeficientes. A diferencia de los modelos de ensemble, la Regresión Logística expone la dirección y magnitud de cada relación: un coeficiente positivo en una variable indica que su aumento favorece la predicción de victoria para player1, y viceversa. Esto permite validar que el modelo está aprendiendo relaciones coherentes con el dominio del tenis.

4.3 Random Forest con Grid Search

El segundo modelo es un Random Forest, un ensemble de árboles de decisión entrenados con bagging. Su principal ventaja sobre la Regresión Logística es la capacidad de capturar relaciones no lineales y variables de interacción entre features sin necesidad de especificarlas explícitamente.

Se aplica Grid Search con validación cruzada para encontrar la combinación óptima de hiperparámetros. El espacio de búsqueda cubre cinco hiperparámetros con los valores siguientes:

Hiperparámetro	Valores explorados
n_estimators	100, 200
max_depth	10, 20, None
min_samples_split	2, 10
min_samples_leaf	1, 4
max_features	sqrt, log2

Tabla 4.2 — Espacio de búsqueda del Grid Search para Random Forest.

El producto cartesiano de estas combinaciones genera 48 configuraciones distintas. Con 5 folds de validación cruzada, el proceso ejecuta un total de 240 ajustes del modelo. La métrica de optimización es el AUC-ROC, consistente con el objetivo de maximizar la capacidad discriminativa del clasificador más allá del simple umbral de 0,5.

Los hiperparámetros óptimos encontrados por el Grid Search son max_depth=20, min_samples_leaf=4, min_samples_split=2, n_estimators=100 y max_features=sqrt. La profundidad máxima de 20 combinada con min_samples_leaf=4 ofrece un equilibrio entre expresividad del árbol y regularización implícita.

4.4 XGBoost con Grid Search

El tercer modelo es XGBoost, un algoritmo de gradient boosting que entrena árboles de forma secuencial, donde cada árbol corrige los errores del anterior. A diferencia del Random Forest, que construye árboles en paralelo e independientes, XGBoost optimiza directamente una función de pérdida mediante descenso por gradiente.

El espacio de búsqueda cubre cinco hiperparámetros:

Hiperparámetro	Valores explorados
n_estimators	100, 200
max_depth	3, 6, 9

learning_rate	0.01, 0.1, 0.3
subsample	0.8, 1.0
colsample_bytree	0.8, 1.0

Tabla 4.3 — Espacio de búsqueda del Grid Search para XGBoost.

El producto cartesiano de estas combinaciones genera 72 configuraciones. Con 5 folds de validación cruzada, el proceso ejecuta 360 ajustes del modelo, el proceso más costoso computacionalmente del pipeline. La función de pérdida interna es logloss, adecuada para clasificación binaria con probabilidades calibradas.

Los hiperparámetros óptimos encontrados son n_estimators=100, max_depth=6, learning_rate=0.1, subsample=0.8 y colsample_bytree=0.8. La profundidad de 6 es conservadora respecto a los valores explorados, y las tasas de submuestreo de 0,8 en ambos parámetros introducen aleatoriedad controlada que actúa como regularización implícita.

5. Evaluación del modelo

Una vez entrenados los tres modelos, se evalúa su rendimiento sobre el conjunto de test, que contiene partidos de las temporadas 2021 a 2024, ninguno de los cuales ha sido visto durante el entrenamiento. Esta sección presenta las métricas individuales de cada modelo, su comparativa, el análisis de la importancia de variables y la selección del modelo final.

5.1 Métricas de evaluación

Se utilizan tres métricas principales para evaluar el rendimiento de los clasificadores:

- **Accuracy:** proporción de predicciones correctas sobre el total. Es la métrica más intuitiva, pero en clasificación binaria puede ser engañosa si las clases están desbalanceadas. En este caso, el dataset simetrizado garantiza un balance perfecto al 50/50, lo que hace que la accuracy sea comparable a la del baseline de predicción por mayoría.
- **AUC-ROC:** área bajo la curva ROC. Mide la capacidad del modelo para discriminar entre las dos clases a lo largo de todos los umbrales posibles de decisión, independientemente del punto de corte elegido. Es la métrica principal de optimización del proyecto y la que mejor refleja la calidad del modelo como ordenador de probabilidades.
- **F1-score macro:** media armónica de precisión y recall, calculada por separado para cada clase y promediada. Permite verificar que el modelo no sacrifica el rendimiento en una clase para mejorar en la otra, lo que sería un síntoma de sesgo posicional no corregido por la simetrización.

Como referencia de contexto, el baseline trivial —predecir siempre al jugador con mejor ranking— alcanza una accuracy del 65,2% sobre el dataset de test, sin ningún modelo. Cualquier clasificador que no supere ese umbral es funcionalmente equivalente a una heurística simple.

5.2 Comparativa de modelos

La tabla siguiente recoge los resultados de los tres modelos evaluados sobre el conjunto de test:

Modelo	Accuracy	AUC-ROC	F1 macro
Regresión Logística	69,38%	0,7621	0,69
Random Forest	69,09%	0,7607	0,69
XGBoost	69,47%	0,7652	0,69

Tabla 5.1 — Métricas de evaluación sobre el conjunto de test (temporadas 2021–2024).

El primer resultado relevante es que los tres modelos convergen en un rango muy estrecho: entre 69,09% y 69,47% de accuracy, y entre 0,7607 y 0,7652 de AUC-ROC. Esta convergencia no es un fallo del proceso de optimización, sino información sobre el techo

predictivo de las features disponibles. Añadir complejidad algorítmica sin añadir información nueva no mejora el rendimiento de forma sustancial.

El segundo resultado relevante es que la Regresión Logística, el modelo más simple de los tres, alcanza el segundo mejor AUC-ROC (0,7621) y el segundo mejor accuracy (69,38%), prácticamente igualando a XGBoost. Esto indica que las relaciones entre las features y la variable objetivo son predominantemente lineales con las features actuales. El Random Forest, a pesar de su mayor capacidad para capturar no linealidades, no logra explotar relaciones que la Regresión Logística no pueda aproximar.

El F1-score macro de 0,69 es idéntico en los tres modelos y coincide con la accuracy redondeada, lo que confirma que ninguno de los clasificadores presenta sesgo hacia ninguna de las dos clases. La simetrización del dataset ha funcionado como se esperaba: el modelo predice victory con la misma fiabilidad que predice derrota.

Comparativa con el sistema de referencia de la ATP

El sistema de predicción oficial de la ATP alcanza una accuracy aproximada del 76%, frente al 69,5% del mejor modelo de este proyecto. La brecha de algo más de seis puntos porcentuales es significativa y tiene una explicación estructural: el sistema de la ATP tiene acceso a variables que no están disponibles en ningún dataset público. Entre ellas, el estado físico de los jugadores, la fatiga acumulada en la semana de torneo, el historial head-to-head reciente en superficie específica, y posiblemente datos de práctica y preparación. Ninguna de estas variables puede derivarse de los registros históricos de partidos.

Esta brecha no invalida el modelo construido en este proyecto. Confirma que el límite superior alcanzable con información pública pre-partido está próximo al 70% de accuracy, y que el proyecto ha llegado a ese techo con un pipeline técnicamente riguroso.

5.3 Feature importance y análisis de coeficientes

El análisis de la importancia de variables revela patrones consistentes entre los tres modelos y permite validar que el pipeline está aprendiendo relaciones coherentes con el dominio del tenis.

En los tres modelos, la variable más predictiva es winrate_diff, la diferencia de winrate bayesiano entre los dos jugadores. En la Regresión Logística tiene el coeficiente más alto (0,98), en el Random Forest acumula el 24% de la importancia total, y en XGBoost el 43%. Este resultado valida el diseño de la feature: capturar el estado de forma reciente con una ventana deslizante de 30 partidos y un estimador bayesiano aporta información que el ranking solo no recoge.

En XGBoost, sin embargo, la concentración en una única variable es excesiva. Con el 43% de la importancia total, winrate_diff prácticamente duplica su peso respecto al Random Forest. Los árboles de boosting tienden a explotar agresivamente las variables más discriminativas, lo que produce modelos más frágiles ante jugadores con historial reciente escaso o atípico.

Las variables de ranking —rank_diff, rank_pts_diff, y en algunos modelos los valores absolutos de ranking de cada jugador— aparecen entre las más importantes en los tres clasificadores, pero fragmentadas. En el Random Forest, seis variables de ranking separadas acumulan una importancia combinada de aproximadamente 0,35, superando a

winrate_diff en conjunto pero diluida entre variables correlacionadas entre sí. Esta fragmentación puede generar inestabilidad en los árboles y reduce la eficiencia del modelo.

En la Regresión Logística, los coeficientes de player1_entry_PR y player2_entry_PR alcanzan valores de aproximadamente 0,77, los segundos más altos del modelo tras winrate_diff. La categoría PR corresponde a jugadores que regresan de lesión con un ranking protegido. Que el modelo les asigne tanto peso merece revisión: si el volumen de partidos con esta categoría es reducido, estos coeficientes pueden estar capturando ruido estadístico en lugar de una relación genuina.

Las variables de mano de juego y de tipo de entrada al torneo aparecen con coeficientes superiores a rank_diff en la Regresión Logística, lo que es contraintuitivo. Una diferencia de ranking debería ser más predictiva que si un jugador es zurdo. Esto sugiere que estas variables están actuando como proxies de información no capturada directamente por las features de rendimiento.

5.4 Curvas ROC

Las curvas ROC de los tres modelos muestran perfiles muy similares, coherentes con la convergencia de los valores de AUC. Los tres clasificadores mejoran significativamente sobre la diagonal de referencia (AUC=0,5, equivalente a predicción aleatoria) en toda la región de la curva, con mayor separación en la zona de bajo false positive rate, donde los modelos son más conservadores en sus predicciones de victoria.

XGBoost presenta la curva ligeramente más alta en la región central, lo que explica su ventaja de 0,003–0,005 puntos de AUC sobre los otros dos modelos. La diferencia es pequeña pero consistente: el gradient boosting captura relaciones que los otros dos algoritmos no alcanzan con las features actuales.

5.5 Selección del modelo final

El modelo seleccionado como resultado final del proyecto es XGBoost, por ser el que obtiene las mejores métricas en todas las dimensiones evaluadas: mayor AUC-ROC (0,7652), mayor accuracy (69,47%) y el mismo F1-score que los otros dos modelos.

La ventaja de XGBoost sobre los otros modelos es modesta en términos absolutos, pero consistente. Tras 360 ajustes de Grid Search, el modelo ha encontrado una configuración que extrae el máximo rendimiento posible de las features disponibles. Sus hiperparámetros óptimos reflejan un diseño conservador: profundidad media (max_depth=6), tasas de submuestreo del 80% y un learning rate moderado (0,1), todo ello orientado a reducir el sobreajuste.

No obstante, hay dos matices importantes sobre este resultado. Primero, la degradación entre el AUC en validación cruzada (0,7731) y en test (0,7652) —un gap de 0,0079— indica la presencia de cierto sobreajuste, probablemente amplificado porque StratifiedKFold no garantiza que el orden temporal se respete dentro de cada fold. TimeSeriesSplit sería más correcto para este tipo de datos y podría reducir ese gap. Segundo, la dependencia excesiva del modelo en winrate_diff (43% de importancia) lo hace potencialmente frágil ante jugadores con poco historial reciente, un escenario frecuente con jugadores jóvenes o en regreso de lesión.

Modelo seleccionado: XGBoost

Accuracy en test:	69,47%
AUC-ROC en test:	0,7652
AUC-ROC en CV:	0,7731
F1-score macro:	0,69
Referencia ATP:	~76% accuracy
Baseline trivial:	65,2% (predecir siempre al favorito por ranking)

En conjunto, los tres modelos demuestran que es posible superar de forma consistente el baseline trivial del 65,2% con un pipeline de features bien diseñado. El margen de mejora respecto al sistema de la ATP refleja la limitación estructural de trabajar solo con información pública, no un fallo del proceso de modelado.

6. Visualización de datos

La componente de visualización del proyecto se desarrolló en Python, dentro del mismo ecosistema técnico que el resto del pipeline. La decisión de no utilizar una herramienta externa de Business Intelligence responde a dos criterios: mantener todo el trabajo en un entorno reproducible y demostrar dominio sobre las capacidades de visualización e interactividad del ecosistema Python.

El resultado es un notebook interactivo con tres páginas de análisis, construido sobre ipywidgets para la interactividad y matplotlib para la generación de gráficos. Es compatible con Google Colab, lo que garantiza su ejecutabilidad sin dependencias de entorno local.

6.1 Objetivos de la visualización

La visualización cubre una función distinta a la del modelo predictivo. Mientras el modelo responde a la pregunta de quién va a ganar un partido concreto, el dashboard responde a preguntas exploratorias sobre los datos históricos del circuito ATP: cómo se distribuyen los resultados por superficie, quién domina en cada condición, qué ocurre cuando dos jugadores se enfrentan directamente, o qué diferencias hay en el juego entre superficies.

Los objetivos específicos de la visualización son tres: ofrecer una visión general del circuito ATP con los indicadores más relevantes del dataset, permitir el análisis head-to-head entre cualquier par de jugadores con partidos registrados, y explorar el comportamiento del circuito en función de la superficie de juego.

6.2 Arquitectura del código

El desarrollo se estructura en dos archivos con responsabilidades claramente separadas, siguiendo buenas prácticas de ingeniería de software.

El primero, viz_atp.py, es un módulo Python que encapsula toda la lógica en 16 funciones organizadas en dos niveles. Las funciones privadas, identificadas con el prefijo de guión bajo, se encargan del renderizado de cada visualización concreta. Las funciones públicas —pagina1_kpis, pagina1_rankings, pagina1_tablas, pagina2_h2h, pagina3_superficie_overview y pagina3_metricas_ranking— gestionan los widgets interactivos y orquestan las llamadas a las funciones de renderizado. Esta separación mantiene el código modular y permite modificar una visualización concreta sin afectar al resto del sistema.

El segundo, viz_atp_notebook.ipynb, es el notebook de presentación. Cada celda de análisis consiste en una única llamada a una función del módulo, sin lógica propia. El resultado es un notebook completamente limpio donde el código visible es mínimo y el foco está en las visualizaciones.

6.3 Preprocesamiento para visualización

El módulo de visualización realiza su propia carga de datos de forma independiente al pipeline del modelo. Trabaja sobre el dataset original de Sackmann en formato ganador/perdedor, sin simetrizar, ya que el objetivo es el análisis exploratorio y no la predicción.

La función de carga aplica los siguientes pasos: carga y concatenación de todos los CSV de partidos del tour principal excluyendo futures, qualifiers, challengers y dobles; filtrado al rango 1968–2024; limpieza de la columna de superficie imputando los nulos como Unknown; y cálculo de métricas derivadas como aces totales por partido y porcentajes de efectividad de saque. A diferencia del pipeline del modelo, esta carga trabaja sobre el rango completo desde 1968, aunque se incluye una nota de cobertura en las visualizaciones que muestran estadísticas de golpe, que tienen nulos significativos antes de 1991.

6.4 Estructura del dashboard — tres páginas

El dashboard se organiza en tres páginas con objetivos analíticos diferenciados. Cada página está diseñada para responder a un tipo de pregunta distinto sobre los datos históricos del circuito ATP.

Página 1 — Resumen general

La primera página ofrece una visión panorámica del circuito ATP a lo largo de todo el período cubierto por el dataset. Se estructura en tres bloques de contenido, todos ellos reactivos a dos filtros globales: un slider de rango de años y un dropdown de superficie que permiten acotar el análisis a cualquier período o condición de juego.

El primer bloque presenta cuatro KPIs en tarjetas visuales que resumen el estado global del dataset bajo los filtros activos: total de partidos, jugadores únicos identificados, duración media de partido en minutos y año con mayor volumen de partidos registrados. Acompañado de un gráfico de anillo con la distribución porcentual de partidos por superficie, que cambia dinámicamente al modificar el filtro de años.

El segundo bloque sitúa en paralelo dos rankings: el top 15 de jugadores por número de victorias acumuladas en el circuito y el top 10 por aces totales a lo largo de su carrera. Ambos se presentan en barras horizontales con etiquetas de valor, lo que facilita la comparación visual directa entre jugadores.

El tercer bloque contiene dos tablas analíticas. La primera recoge todas las finales de Grand Slam del período, filtrable por rango de años, con columnas de torneo, superficie, ganador, finalista y resultado. La segunda es una tabla de upsets históricos con un slider independiente que permite al usuario definir el gap mínimo de ranking entre ganador y perdedor para considerar un resultado como sorpresa. Esta funcionalidad permite explorar desde upsets moderados hasta las victorias más improbables del período.

Elemento	Tipo	Filtros disponibles
KPIs (4 indicadores)	Tarjetas	Rango años · Superficie
Distribución por superficie	Gráfico de anillo	Rango años
Top 15 victorias	Barras horizontales	Rango años · Superficie
Top 10 aces	Barras horizontales	Rango años · Superficie
Finales Grand Slam	Tabla paginada	Rango años
Upsets históricos	Tabla con slider	Gap mínimo de ranking

Tabla 6.1 — Elementos de la Página 1 y filtros disponibles en cada uno.

Página 2 — Head to Head

La segunda página implementa el análisis de enfrentamientos directos entre dos jugadores. Es la página técnicamente más compleja del dashboard, ya que requiere filtrado dinámico en cadena y el cálculo de estadísticas específicas para cada par de jugadores seleccionados.

La selección de jugadores se realiza mediante dos dropdowns encadenados. Al elegir el primer jugador, el segundo dropdown se reconstruye dinámicamente mostrando únicamente los rivales contra los que existen partidos registrados en el dataset. Esta actualización es instantánea porque se apoya en un diccionario precalculado al arrancar el módulo que mapea cada jugador a su conjunto de rivales reales, evitando recalcular sobre el dataset completo en cada interacción.

Una vez seleccionados ambos jugadores, el dashboard despliega cinco bloques de información. El primero muestra el marcador global de victorias de cada jugador acompañado de una barra de porcentaje proporcional que visualiza el dominio relativo en el historial. El segundo desglosa el marcador por superficie en tres bloques independientes —Hard, Clay y Grass— con marcador individual y porcentaje por condición de juego, lo que permite detectar si uno de los jugadores tiene ventaja específica en alguna superficie.

El tercer bloque presenta un gráfico de barras agrupadas con las victorias de cada jugador por superficie, complementando la lectura numérica con una comparación visual directa. El cuarto bloque es una tabla de estadísticas comparativas de golpe por partido: aces, dobles faltas, porcentaje de primer servicio, puntos ganados con primer y segundo servicio, y break points salvados. Estas estadísticas se calculan combinando las filas en las que cada jugador actuó como ganador y como perdedor, para obtener promedios que reflejen su juego real independientemente del resultado. El quinto bloque es el historial completo de partidos entre ambos jugadores, con columnas de año, torneo, superficie, ronda y resultado.

Página 3 — Análisis por superficie

La tercera página explora el comportamiento del circuito y de los jugadores en función de la superficie de juego. Se organiza en dos bloques complementarios.

El primer bloque presenta una visión comparativa entre superficies. Tres tarjetas independientes —una por superficie principal— muestran el número absoluto de partidos y su peso porcentual sobre el total del dataset. A continuación, un gráfico de barras horizontales apiladas muestra las victorias del top 10 de jugadores desglosadas por Hard, Clay y Grass, lo que permite identificar de forma visual si el dominio de un jugador es generalista o está concentrado en una superficie específica.

El segundo bloque profundiza en las diferencias técnicas entre superficies. Tres gráficos de barras individuales comparan el porcentaje de primer servicio ganador, la duración media de partido en minutos y los aces por partido para cada superficie. Dado que las estadísticas de golpe tienen cobertura de datos limitada antes de 1991, se incluye una nota de cobertura explícita en cada gráfico para contextualizar el alcance del análisis.

El bloque se completa con una tabla de ranking filtrable por superficie. Para cada jugador con un mínimo de 20 partidos disputados en la superficie seleccionada, la tabla muestra victorias, partidos totales, winrate y aces por partido. El umbral de 20 partidos garantiza que los jugadores incluidos tienen muestra suficiente para que sus estadísticas sean representativas.

Página	Pregunta que responde
1 — Resumen general	¿Cómo es el circuito ATP en el período y superficie seleccionados?
2 — Head to Head	¿Qué ocurre cuando estos dos jugadores se enfrentan?
3 — Análisis por superficie	¿Cómo cambia el juego y el dominio según la superficie?

Tabla 6.2 — Objetivo analítico de cada página del dashboard.

Visualización — resumen técnico

Tecnología: Python · ipywidgets · matplotlib
Entorno: Google Colab (sin dependencias locales)
Arquitectura: módulo viz_atp.py + notebook de presentación
Funciones: 16 (8 privadas de renderizado + 8 públicas de orquestación)
Páginas: 3 (Resumen general / Head to Head / Análisis por superficie)
Datos: dataset original Sackmann 1968–2024 (sin simetrizar)

7. Gestión del proyecto

7.1 Metodología aplicada

El proyecto se ha desarrollado siguiendo la metodología CRISP-DM (Cross-Industry Standard Process for Data Mining), el marco de referencia más extendido en proyectos de ciencia de datos. CRISP-DM organiza el trabajo en seis fases iterativas que no siguen un orden estrictamente lineal: la comprensión del negocio, la comprensión de los datos, la preparación de los datos, el modelado, la evaluación y el despliegue.

La naturaleza iterativa de CRISP-DM es especialmente adecuada para este proyecto. La fase de comprensión de los datos —el EDA— generó decisiones que retro-alimentaron la fase de preparación: detectar outliers de altura, descubrir el sesgo posicional ganador/perdedor o identificar el data leakage en las estadísticas in-match son hallazgos que surgieron durante la exploración y que modificaron el diseño del preprocesamiento. Del mismo modo, los resultados de la evaluación de los modelos han identificado líneas de mejora concretas que retroalimentarían una siguiente iteración del pipeline.

El desarrollo se apoyó en Google Colab como entorno de ejecución, con guardado intermedio de los datasets procesados en Google Drive para evitar recalcular el pipeline completo en cada sesión. El control de versiones del notebook se gestionó manualmente mediante nombrado incremental de archivos.

7.2 Alcance, tiempo y coste

Alcance

El alcance del proyecto cubre tres entregables: el pipeline de preprocesamiento y modelado implementado en Python (notebook principal), el módulo de visualización interactiva (viz_atp.py y notebook de presentación), y la presente memoria. Quedan fuera del alcance el despliegue del modelo en producción, la integración con APIs de datos en tiempo real y cualquier componente de predicción sobre partidos futuros más allá del entorno de evaluación académica.

Tiempo

El proyecto se ha desarrollado en un período estimado de seis a ocho semanas a media jornada, superando el mínimo de 40 horas exigido por la rúbrica de evaluación. La distribución aproximada del tiempo por fase es la siguiente:

Fase	Dedicación estimada
Comprensión del problema y exploración inicial	6 horas
EDA y auditoría de datos	8 horas
Diseño e implementación del preprocesamiento	12 horas
Feature engineering (winrate bayesiano, simetrización)	10 horas

Entrenamiento y evaluación de modelos	8 horas
Desarrollo del módulo de visualización	10 horas
Redacción de la memoria	8 horas
Total estimado	~62 horas

Tabla 7.1 — Distribución estimada de horas por fase del proyecto.

Coste

El proyecto no ha incurrido en costes económicos directos. Todas las herramientas utilizadas son de acceso gratuito o de código abierto: Python y sus librerías (pandas, scikit-learn, XGBoost, matplotlib, ipywidgets) bajo licencias open source, Google Colab como entorno de ejecución en su modalidad gratuita, y el repositorio de Jeff Sackmann bajo licencia MIT. El coste de oportunidad del tiempo dedicado es el único coste real del proyecto.

Recurso	Coste
Python + librerías (pandas, sklearn, XGBoost, matplotlib...)	0 € (open source)
Google Colab	0 € (modalidad gratuita)
Dataset JeffSackmann/tennis_atp	0 € (licencia MIT)
Herramientas de IA (Claude Sonnet/Opus)	0 € (incluido en suscripción)
Coste total del proyecto	0 €

Tabla 7.2 — Desglose de costes del proyecto.

8. Conclusiones y recomendaciones

Este proyecto ha construido un pipeline completo de ciencia de datos para la predicción de resultados de partidos ATP, desde la carga del dato crudo hasta la evaluación de tres modelos de clasificación, complementado con un módulo de visualización interactiva. Las conclusiones se organizan en torno a los tres ejes principales del trabajo: los resultados del modelo, las decisiones de diseño del pipeline y las líneas de mejora identificadas.

8.1 Conclusiones sobre el modelo

El modelo final, XGBoost, alcanza un AUC-ROC de 0,7652 y una accuracy del 69,47% sobre partidos de las temporadas 2021 a 2024. Estos resultados permiten extraer tres conclusiones principales.

Primera: es posible predecir el resultado de un partido ATP con información pública pre-partido a un nivel significativamente superior al baseline trivial. El baseline de predecir siempre al favorito por ranking alcanza el 65,2%, mientras que el modelo supera ese umbral en más de cuatro puntos porcentuales de accuracy y alcanza un AUC-ROC de 0,76 —lo que implica que el modelo ordena correctamente los dos jugadores en más de tres de cada cuatro pares comparados.

Segunda: la brecha de aproximadamente seis puntos respecto al sistema de predicción de la ATP (~76%) tiene una explicación estructural, no algorítmica. Los tres modelos convergen en un rango de accuracy de apenas 0,4 puntos porcentuales entre el peor y el mejor, lo que indica que el techo predictivo está determinado por la información disponible, no por la elección del algoritmo. Las variables que explicarían esa brecha —estado físico, fatiga, historial head-to-head reciente, condiciones del día— no están presentes en ningún dataset público.

Tercera: la feature más predictiva del pipeline es el winrate bayesiano con ventana deslizante, que acumula entre el 24% y el 43% de la importancia total según el modelo. Esto valida el diseño de esa feature: capturar la forma reciente del jugador con un estimador estabilizado aporta información que el ranking, por sí solo, no recoge. El ranking refleja rendimiento acumulado; el winrate refleja el momento actual.

8.2 Conclusiones sobre el pipeline

El diseño del preprocesamiento ha sido la fase más crítica del proyecto. Tres decisiones han tenido un impacto directo en la validez de los resultados: la eliminación de las 19 columnas de estadísticas in-match para evitar data leakage, la simetrización del dataset para eliminar el sesgo posicional inherente al formato ganador/perdedor, y el uso de un split temporal en lugar de un split aleatorio para que la evaluación refleje condiciones reales de uso.

Sin estas tres decisiones, el modelo habría producido métricas infladas sin valor predictivo real. La accuracy que el modelo habría obtenido incluyendo estadísticas in-match habría superado el 90%, un número que no tiene ninguna aplicabilidad práctica porque esas variables no existen en el momento de la predicción. La simetrización ha sido igualmente crítica: la matriz de confusión perfectamente simétrica de la Regresión Logística confirma que el sesgo posicional ha sido eliminado con eficacia.

8.3 Recomendaciones para iteraciones futuras

La evaluación de los modelos ha identificado varias líneas de mejora concretas que aumentarían el rendimiento predictivo en una siguiente iteración del proyecto.

La más prometedora es la incorporación de una feature de historial head-to-head entre cada par de jugadores, calculada con la misma lógica de ventana temporal que el winrate bayesiano para evitar data leakage. El enfrentamiento directo previo entre dos rivales tiene valor predictivo reconocido en tenis, especialmente en superficie específica, y su ausencia es una limitación estructural del pipeline actual.

Una segunda mejora sería el cálculo de winrates por superficie: en lugar de un único winrate general, calcular tasas de victoria específicas para Hard, Clay y Grass capturaría la especialización de jugadores como Rafael Nadal en tierra batida o el perfil diferente que presentan muchos jugadores según las condiciones. Esta feature también resolvería parcialmente la presencia anómala de surface_Clay como único representante de superficie en el top de importancia de XGBoost.

En el plano metodológico, sustituir StratifiedKFold por TimeSeriesSplit en la validación cruzada del Grid Search reduciría el gap entre el AUC en validación y en test, ya que garantizaría que el orden temporal se respeta estrictamente dentro de cada fold y no solo entre folds. La degradación observada de 0,0079 puntos de AUC entre CV y test en XGBoost apunta a que la validación cruzada actual ve datos ligeramente del futuro dentro de cada iteración interna.

Finalmente, la consolidación de las variables de ranking en una o dos features derivadas —como el ratio de puntos o la diferencia normalizada por el total de puntos en juego— reduciría la multicolinealidad entre las seis variables de ranking actuales y podría mejorar la estabilidad de los modelos de árbol.

Resumen de conclusiones

Modelo final: XGBoost · AUC-ROC 0,7652 · Accuracy 69,47%
Baseline trivial: 65,2% (predecir al favorito por ranking)
Referencia ATP: ~76% (con variables no disponibles públicamente)

Feature más predictiva: winrate_diff (43% importancia en XGBoost)
Decisiones clave: eliminación in-match · simetrización · split temporal

Principal mejora pendiente: feature head-to-head + winrate por superficie

9. Referencias

Sackmann, J. (2024). tennis_atp [Repositorio de datos]. GitHub. https://github.com/JeffSackmann/tennis_atp

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794. https://doi.org/10.1145/2939672.2939785

McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 56–61.

Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324

Wirth, R., & Hipp, J. (2000). CRISP-DM: Towards a Standard Process Model for Data Mining. Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining, 29–39.

Fawcett, T. (2006). An introduction to ROC analysis. Pattern Recognition Letters, 27(8), 861–874. https://doi.org/10.1016/j.patrec.2005.10.010

Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

10. Anexos

10.1 Notebook completo (pipeline Python)

Se adjunta el notebook Preprocesing_ATP_BD.ipynb con el pipeline completo de preprocesamiento, feature engineering y modelado. El notebook está comentado siguiendo las convenciones PEP8 y organizado en cuatro fases marcadas con encabezados: Setup e importaciones, Preprocesamiento, Feature Engineering y Modelado.

El notebook es autocontenido: clona el repositorio de Sackmann directamente desde GitHub y ejecuta el pipeline completo desde cero. El único requisito externo es una cuenta de Google Drive para el guardado del dataset intermedio con el winrate calculado.

10.2 Módulo de visualización

Se adjuntan los archivos viz_atp.py y viz_atp_notebook.ipynb correspondientes al módulo de visualización interactiva. El notebook de presentación es ejecutable en Google Colab sin modificaciones.

10.3 Prompts e IA utilizados en el proyecto

A continuación se recoge el listado completo de usos de herramientas de inteligencia artificial durante el desarrollo del proyecto, con los prompts representativos de cada categoría de uso.

Herramienta utilizada: Claude Sonnet 4.6 y Claude Opus 4.6 (Anthropic), accedidos a través de Claude.ai y Claude Code.

Categoría de uso	Prompts representativos
Corrección de errores en el pipeline	"El merge del winrate bayesiano produce NaN en algunas filas tras la simetrización. Revisa el código e identifica dónde se pierde la asignación." / "El StandardScaler está dando error al aplicarse sobre columnas con NaN. Identifica qué columnas tienen nulos residuales tras el preprocesamiento."
Comentado y documentación del código	"Añade docstrings PEP8 a esta función de simetrización explicando cada parámetro y el valor de retorno." / "Revisa los comentarios en línea de este bloque de feature engineering y mejóralos para que sean autoexplicativos."
Apoyo en la redacción de la memoria	"Redacta la sección de preprocesamiento de datos siguiendo este esquema y estos datos reales del pipeline [...]." / "Reescribe este párrafo sobre data leakage en tono académico sin fragmentos de código."

Tabla 10.1 — Uso de herramientas de IA por categoría con prompts representativos.

En ningún caso se utilizó la IA para tomar decisiones técnicas sobre el diseño del modelo, la selección de features, la interpretación de resultados ni la evaluación del rendimiento de los algoritmos. El uso se limitó estrictamente a soporte en depuración de código, mejora de la documentación y redacción de la memoria.